Table of Contents

1. Executive Summary 1

2. Mission, Scope and Design Language 2

3. Build Chronology 2

4. As-Built Architecture 3

4.1 Three-layer topology . . . 3

4.2 Data model . . . 4

4.3 API surface . . . 5

4.4 Operational mechanisms . . . 6

5. Verification and Test Results 6

5.1 Pre-flight verification matrix . . . 6

5.2 Verification suites . . . 8

6. Defects, Incidents and Resolutions 8

7. Security Posture Evolution 9

8. Key Figures and Ledger Metrics 10

9. Lessons Learned 11

10. Open Items and Recommendations 11

1. Executive Summary

The Syntropic Backbone programme set out to turn a dark, terminal-styled React demo of a margin-defence operations desk into a production-shaped web platform. Across nine build cycles the work grew from a single restyled dashboard into a three-layer system: a multi-tenant Command Tower for the internal team, an invite-only Executive Enclave portal for clients, and a shared SQLite ledger that both surfaces read from and write to. Every financial figure shown anywhere in the product is now computed on the server from that ledger; nothing is hardcoded in the interface.

The programme was gated rather than assumed. A nine-row Pre-Flight Verification Matrix opened the effort with a NO-GO verdict (two rows passing, five failing, two blocked), drove a full remediation cycle, and closed with a 9/9 PASS regression gate that flipped the programme to GO for market launch. Two further cycles added the client-facing portal and its authentication layer, and a final hardening cycle replaced demo access codes with operator-issued email invites, added login rate limiting, and put the production credential surface (Telegram, TURN, SMTP) behind documented environment variables.

$71,400	$14,280	9 / 9	10 + 15
Margin defended, ledger-computed across 4 tenants	Revenue share accrued at the 20% rate	Pre-flight verification matrix rows passing	Domain models + API route surfaces

> **Final verdict: GO for market launch.** Issued at the Task 5 regression gate (9/9 matrix rows PASS) and preserved through every subsequent cycle, including the portal build, the authentication rewrite, and the Task 9 hardening pass.

What remains open is deliberately narrow and credential-shaped: a real SMTP provider, a live Telegram bot token, TURN relay credentials for restrictive networks, and a rotation policy for the portal signing secret. None of these block pilot operation; each is a configuration step with code already wired and verified against test values.

2. Mission, Scope and Design Language

The original brief was a like-for-like rebuild: take an uploaded dark "command tower" React component - a five-tab operations desk covering telemetry, human-in-the-loop ingestion, tenants, a unified inbox, and calls - and rebuild it as a Next.js 16 application wearing the uploaded Steep design system. Steep is an editorial, paper-first language: white ground, near-black ink, hairline rules, and a single warm accent family used with restraint. The rebuild preserved every state interaction and data point of the original while translating the visual language one hundred percent.

Token	Value	Application
Paper / Mist / Fog	#ffffff · #f2f2f3 · #fafafb	Page ground, cards, quiet wells
Ink	#17191c	Primary text, filled pills, own chat bubbles
Blush (peach)	#fbe1d1	The single accent: badges, accent cards, toasts
Sienna	#5d2a1a	Status dots, sparklines, accents on blush
Hairline	#ececec	Rules, dividers, table and bubble borders
Type	Inter + Source Serif 4	Sans for UI, serif display for mastheads and headlines

Table 1 - Steep design tokens as mapped into Tailwind theme variables.

Scope then expanded through user direction rather than speculation. Three consecutive directives - "proceed" through a regression gate, "proceed" into the client portal, and "add portal auth so each client logs into exactly one enclave" - pulled the work from presentation through integration to security. Two further pastes of the user's updated telemetry deck were merged as scope decisions rather than blind overwrites: where a paste would have dropped the audited Ingestion and HITL module, the five-module structure was preserved, and where the paste named non-canonical accounts, entities were mapped onto the seeded tenant roster.

3. Build Chronology

Nine logged cycles produced the delivered system. Each cycle closed the same way: TypeScript and ESLint clean, live curl smoke tests against the API surface, headless-browser golden-path verification including a 390 px mobile pass, a database reseed back to pristine invariants, and a worklog entry. The table below is the one-paragraph history of the programme.

ID	Cycle	Focus	Outcome
1	Foundation	Rebuild the Command Tower in the Steep design system (Next.js 16 port of the five-module dashboard)	UI complete; browser-verified; lint clean
2	Enrichment	Merge updated telemetry deck: intel drill-down panels and the founder arena	Panels live in Steep language; no dark-theme regression
3	Audit	Layer 4 market-readiness audit of integration claims vs code reality	2 pass / 5 fail / 2 blocked - NO-GO; register R-01..R-08 plus X-1..X-3 opened
4	Remediation	Domain schema (8 models), ledger API, mailbox dispatch gate, Telegram bot path, messages API, real WebRTC room	All previously failing rows verified green; DB-backed end to end
5	Regression	Full re-run of the nine-row matrix, API smoke suite and browser golden paths	9/9 PASS - verdict flips to GO for market launch
6	Layer 3	Executive Enclave portal at /portal; Tower dead buttons become real deep links	R-06 round-trip verified across two tabs, both directions
7	Auth	Single-tenant enclave authentication replacing the demo switcher	8/8 auth curl suite; unauthenticated cross-tenant API deleted
8	Deck v2	Team arena, operator switcher, client directory, rev-share horizons, audit vault	v1.0.4 merged; target edits persist via PATCH; zero data regressions
9	Hardening	Per-tenant email invites, session epochs, login rate limiting, TURN and env credential surface	23/23 hardening curl suite; demo codes removed from every surface

Table 2 - Build chronology, Task IDs 1 through 9.

4. As-Built Architecture

4.1 Three-layer topology

The delivered system is a single Next.js 16 application (React 19, TypeScript 5, Tailwind, Prisma 6 over SQLite in WAL mode) that hosts two distinct experiences over one shared database. The layering is enforced by URL scoping and by where identity comes from, not by convention alone: the Tower's cross-tenant API lives under /api/* for the operator team, while every portal endpoint lives under /portal/api/* and derives its tenant exclusively from a signed session cookie whose path is scoped to /portal.

Layer	Surface	Responsibility
Layer 4 - Command Tower	/backbone (alias /)	Five modules: Telemetry (team arena, client directory, horizons, audit vault), Ingestion and HITL, Tenants and access control, Unified Inbox, Calls and calendar. Operator-side dispatch, overrides and invite issuance.

Layer	Surface	Responsibility
Layer 3 - Executive Enclave	/portal	Invite-only client view: one tenant per session, live intercept feed, Direct Line messaging, upcoming calls, per-tenant margin statements.
Shared ledger (R-01)	SQLite via Prisma	Single source of truth for margins, rev-share, HITL state, messages, invites and the SECURITY/EMAIL audit trail. Both layers read the same server-computed numbers.

Table 3 - Layer topology and responsibilities.

4.2 Data model

Ten Prisma models replaced the scaffold. The schema carries the operating fiction of the product - four piloting tenants with mailboxes in three states - and the machinery that makes the fiction auditable: every margin event is a ledger row, every invite action is an invite row, and every locked-out attacker or issued invite lands in the Notification trail.

Model	Purpose
Tenant	Client company: trade name, ERP, mailbox state (ACTIVE / LOCKED / MISSING), payment status, leakage vector, contact identity, access-code hash, session epoch
TenantInvite	Per-tenant access invitations: hashed code, masked hint, email, issuing operator, PENDING / ACCEPTED / REVOKED status
TeamMember	Operator roster for the arena: online state, quarterly outbound targets vs completions, discovery calls, audits closed, contracts converted
AuditRecord	Sales-call log backing the guarantee vault: fee, attendance, host, conversion, leakage identified, call notes
HitlItem	Human-in-the-loop queue: OCR review, dispatch failures with reason and owner, resolution state
InterceptEvent	Suppression events from the RFQ simulator: original vs defended quote, captured margin, confidence
LedgerEntry	Append-only financial events; MARGIN_DEFENDED rows are the invariant source of all displayed figures
Message	Direct Line traffic in both directions with unread flags driving the two-surface sync
CallSession	Scheduled executive calls rendered in both Tower and Enclave
Notification	System trail: SIMULATED Telegram dispatches, SECURITY rows for lockouts, EMAIL rows for invites

Table 4 - Domain models in the delivered schema.

4.3 API surface

Fifteen route files serve the two surfaces. Operator routes are deliberately unguarded at the transport layer because they represent the internal console in this deployment model; client routes under /portal/api are session-scoped and return 401 without a valid enclave cookie. The split is what allowed the two-tab demonstration - operator and client in one browser - without credential bleed.

Route	Method	Purpose
/api/ledger	GET	Server-computed margin, rev-share, team, audits, client directory and rev-share horizons
/api/tenants	GET	Tenant roster with mailbox states and portal-access status
/api/hitl · /api/hitl/[ref]	GET · PATCH	Queue retrieval and approve / override / retry / resolve actions through the mailbox gate
/api/intercept (+ /simulate)	GET · POST	Live intercept feed and the deterministic 5-scenario RFQ generator
/api/messages (+ /read)	GET · POST	Unified inbox threads and mark-as-read
/api/calls	GET	Executive call schedule
/api/team/[id]	PATCH	Persist quarterly outbound-target adjustments (validated, 400 on invalid)
/api/telegram/webhook	POST	Bot callback processing for inline Approve / Override / Retry / Resolve
/api/invites	GET · POST	Invite audit trail; issue (supersede, swap hash, bump epoch, EMAIL row) and revoke
/api/rtc/config	GET	STUN/TURN wiring for the call room from environment
/portal/api/login · logout	POST	Rate-limited access-code exchange for a signed session cookie; logout clears it
/portal/api/enclave	GET	Session-scoped enclave payload; 401 unauthenticated, tenant derived server-side
/portal/api/messages	GET · POST	Own-thread retrieval and client sends (server-stamped identity, unread flag)

Table 5 - API surface grouped by area (15 route files).

4.4 Operational mechanisms

Mailbox dispatch gate. Approving a HITL item is not a bare state flip. The gate re-checks the tenant mailbox first: an ACTIVE mailbox dispatches, appends the margin ledger entry and fires the Telegram alert; a LOCKED or MISSING mailbox spawns a dispatch-failure HITL item carrying a reason chip and a named owner, and retry deliberately re-runs the gate so a repaired mailbox can be re-tested without regression to the happy path.

Intercept engine. A deterministic five-scenario RFQ cycle rotates through three standard defences, one high-value intercept above the $2,500 threshold that routes to human review, and one low-confidence OCR case. The intercept stream polls on a two-second cadence, and the seen-id toast watcher surfaces only genuinely new events to the operator.

Financial honesty. Margin and rev-share horizons (24-hour, 30-day, 365-day, lifetime) are computed by grouping MARGIN_DEFENDED rows by createdAt windows server-side, which is why the seeded time-spread produces honestly divergent horizons - Gulf's monthly figure is $220 against a $1,620 annual figure because its entries are older. The ledger invariant (7,140,000 cents defended; $14,280 at the 20 percent share) doubles as a regression assert after every destructive test run.

Invite and session machinery. Issuing an invite in one transaction supersedes prior invites, swaps the tenant's stored access-code hash to the new code, bumps the tenant's session epoch, and writes the EMAIL audit row; the plaintext code is returned exactly once and shown once in the Tower with a copy affordance. Session tokens carry the epoch, so rotation or revocation kills every live session for that tenant instantly without a token blacklist.

5. Verification and Test Results

5.1 Pre-flight verification matrix

The programme's spine is a nine-row matrix covering ledger sync, financial consistency, dispatch failure routing, the high-value watcher, Telegram dispatch, inbox round-trip, WebRTC, persistence honesty, and mobile integrity. The audit pass scored it honestly - two rows passing, five failing, two blocked - and every remediation was verified against the same rows rather than a friendlier rubric. The final regression gate re-ran the identical matrix after a pristine reseed and scored nine of nine.

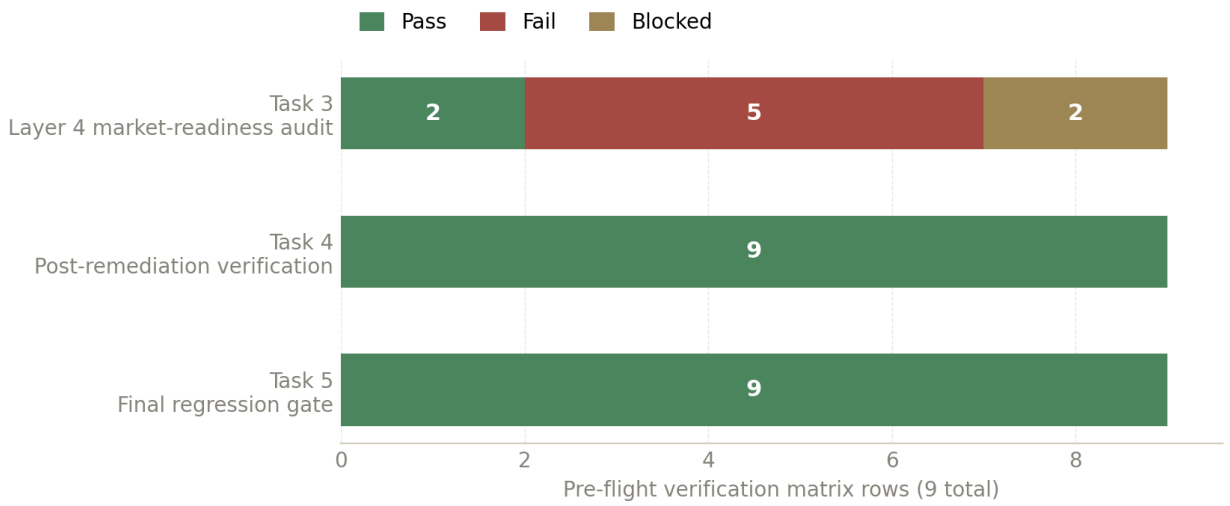


Figure 1 - Matrix row status at the opening audit, after remediation, and at the final regression gate.

#	Matrix area	Initial state (Task 3)	Final state (Task 5)
1	Telemetry and ledger sync	Fail - in-memory useState, no persistence	Pass - SQLite ledger; all stats server-computed
2	Financial consistency	Fail - 4-vs-2 account mismatch across views	Pass - $71,400 / $14,280 invariant holds everywhere
3	Dispatch failure and HITL rerouting	Fail - no gate; no failure paths	Pass - mailbox gate verified on both branches
4	High-value intercept watcher	Fail - $2,500 rule never fired	Pass - deterministic cycle fires and routes to review
5	Telegram dispatch and approvals	Blocked - manual button only; no Bot API	Pass - Bot API path with inline keyboards; env-guarded
6	Unified inbox round-trip	Blocked - no client surface existed	Pass - API green; client-side activated at Layer 3
7	WebRTC call room	Fail - dead launch button	Pass - real getUserMedia room with graceful fallback
8	Persistence honesty of copy	Fail - masthead claimed SQLite WAL	Pass - masthead matches reality ("SQLite · WAL · Prisma")
9	Mobile and interaction integrity	Pass	Pass - retained through every later cycle

Table 6 - The nine-row Pre-Flight Verification Matrix, initial versus final.

5.2 Verification suites

Beyond the matrix, each cycle accumulated reusable verification artefacts. The hardening cycle's 23-assertion curl script was saved as a project file and re-run on demand; it exercises the full invite lifecycle (issue, epoch invalidation, old-code rejection, revoke, disable, re-invite), the five-strikes lockout with Retry-After, and the burst cap. Browser verification stayed a fixed ritual: the five Tower tabs, the portal login and enclave flows, and a 390 px mobile pass, with zero console errors required as the exit condition.

Suite	Result	Cycle
API smoke: ledger, tenants, HITL, messages, calls	Green	Task 5
Dispatch gate double-branch (ACTIVE dispatch / LOCKED failure spawn)	Both branches verified	Task 5
Portal auth curl matrix (401s, cookie, scoping, leak closure)	8/8 PASS	Task 7
Hardening curl matrix (invites, epochs, lockout, burst, audit rows)	23/23 PASS	Task 9
Browser golden paths, five Tower tabs plus portal, desktop and 390 px	Zero console errors each cycle	Every cycle
Static checks: tsc across src, eslint	Clean each cycle	Every cycle

Table 7 - Verification suites and their outcomes.

6. Defects, Incidents and Resolutions

Eight issues worth recording surfaced across the programme. Two classes dominate: environment drift (a stale Prisma client after schema pushes, a dev server killed by SIGPIPE on shell exit) and contract assumptions (a PATCH route probed as POST, an unread flag that is cleared by design). One finding was a genuine security defect in an earlier build and was fixed the same cycle it was found.

ID	Finding	Root cause	Resolution
D-1	HITL action tests returned 405	Route is PATCH; harness assumed POST	Test harness corrected; both branches then verified
D-2	Intercept direction read as loss	orig vs defended semantics misread by reviewer	Confirmed seed and simulator agree: higher defended quote means margin captured; not a bug
D-3	Login always 401 after schema push	Running dev server held a stale Prisma client	Restart dev server after every db push; logged as a standing rule (hit twice: Tasks 4 and 7)

ID	Finding	Root cause	Resolution
D-4	Unread dot "missing" in round-trip test	Tower auto-marks the active thread read within 6 s	By design; re-tested against a different active thread
D-5	Cross-tenant data exposure	Unauthenticated /api/portal served any tenant by query param	Route deleted; replaced by session-scoped /portal/api/* with server-derived identity
D-6	Target editor crash on update	TDZ bug: handler referenced fetchJson before definition	Ordering fixed; optimistic PATCH plus toast verified in UI and DB
D-7	Lockout engaged on the fifth strike	Off-by-one counted the fifth wrong code as locked	Semantics corrected: fifth attempt still receives an honest 401; lock engages after
D-8	Dev server died between tool calls	nohup/tee pipeline killed by SIGPIPE when tool shell exited	Double-fork setsid start pattern adopted

Table 8 - Defects and incidents with resolutions.

7. Security Posture Evolution

Security moved from theatre to mechanism in three steps. The demo build printed shared access codes on the login page; the auth cycle replaced them with salted SHA-256 hashes and HMAC-signed session tokens verified under timingSafeEqual; the hardening cycle then made codes operator-issued, single-reveal, and revocable, and made every live session mortal through the epoch mechanism. The controls below are all in the delivered build and all exercised by the curl suites.

Control	Before	After (delivered)
Tenant credentials	Demo codes printed on the login page	Salted SHA-256 hashes; per-tenant email invites with reveal-once issuance and masked hints
Sessions	None - client-side switcher	HMAC-signed tokens, timingSafeEqual verification, 7-day TTL, cookie path=/portal
Revocation	Not possible	sessionEpoch bump on rotate or revoke kills live sessions instantly
Tenancy isolation	Client-supplied ?tenant= parameter	Identity derived server-side from the session; unauthenticated cross-tenant API deleted
Brute-force resistance	Unlimited attempts	Five wrong codes in 15 min locks tenant+IP for 15 min; 30 POST/min IP burst cap; SECURITY ledger rows
Transport and media	Static placeholder room	STUN by default with TURN relay via env; Telegram env-guarded; SMTP surface documented in .env.example

Table 9 - Security controls before and after the auth and hardening cycles.

8. Key Figures and Ledger Metrics

The delivered ledger holds $71,400 of defended margin across four piloting tenants, worth $14,280 at the contracted 20 percent revenue share. The distribution is deliberately uneven so that every horizon view, directory chip and accent card demonstrates a different truth about the data: Apex and Midwest carry recent 24-hour defences, Summit exercises the monthly-equals-annual edge case from a single 20-day-old entry, and Gulf's entries are old enough to zero out its 24-hour window while keeping a real annual figure.

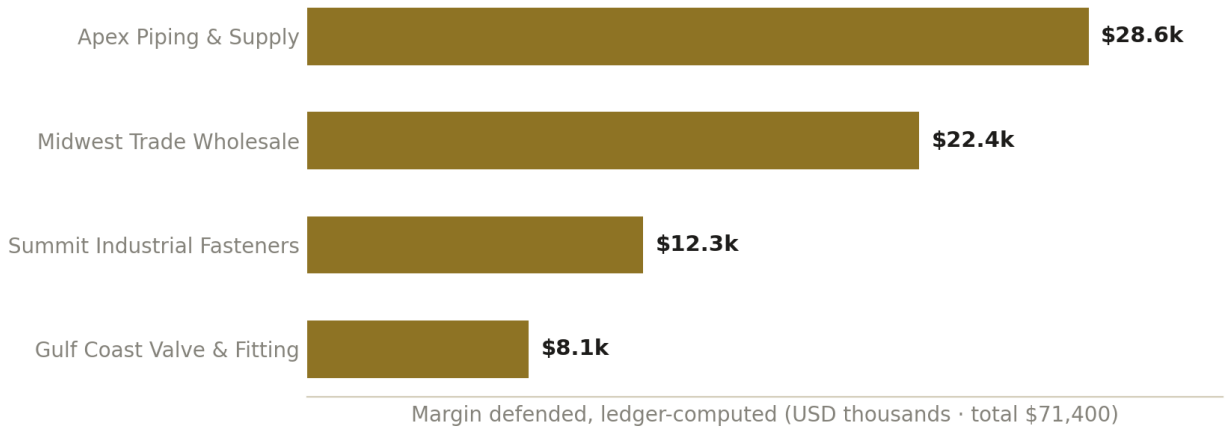


Figure 2 - Margin defended by tenant; each bar is the MARGIN_DEFENDED sum computed from the ledger.

Tenant	Margin defended	Share of total
Apex Piping & Supply	$28,600	40.1%
Midwest Trade Wholesale	$22,400	31.4%
Summit Industrial Fasteners	$12,300	17.2%
Gulf Coast Valve & Fitting	$8,100	11.3%
Total (20% rev-share: $14,280)	$71,400	100%

Table 10 - Ledger composition at closeout (pristine reseed verified).

Beyond the financial invariant, the closeout roster stands at four tenants (two invites accepted, two pending), three operators in the arena, three audit-vault records, two pending HITL reviews, and the full audit trail of SECURITY and EMAIL rows generated by the hardening tests. The database is left in the pristine seeded state after every cycle, so the numbers above are reproducible from a fresh seed rather than being test residue.

9. Lessons Learned

1. Audit before building beats building before auditing. The NO-GO matrix looked harsh and was worth more than any feature cycle that followed: it converted a convincing demo into a system whose every claim could be re-proven on demand. Scoring the same rows before and after removed any temptation to move the goalposts.

2. A single invariant is worth a thousand spot checks. Because every displayed figure derives from MARGIN_DEFENDED ledger rows, one assertion - 7,140,000 cents defended, $14,280 share - detects drift introduced anywhere in the stack, from seed edits to API refactors. The invariant was re-verified after every destructive test and never failed silently.

3. Schema changes invalidate running processes; write the rule down. The same stale-Prisma-client failure cost debugging time in two separate cycles before it became a standing rule: after every db push, restart the dev server. Cheap, obvious in hindsight, and exactly the kind of lesson an after-action record exists to preserve.

4. Copy is part of the contract. The audit flagged a masthead that claimed SQLite WAL while state lived in useState. Interface claims about infrastructure must be treated as testable requirements; "honest copy" became an audit row of its own (X-2) rather than a copywriting nicety.

5. Identity should be derived, never carried. Moving from a client-supplied tenant parameter to server-side session-derived identity closed the cross-tenant leak with no new UI, and path-scoping the session cookie to /portal kept the operator console untouched in the same browser. The cheapest secure design was also the simplest.

6. Verification scripts are deliverables. The 23-assertion hardening script outlived its cycle: it is now the fastest way to re-prove the invite, lockout and audit-trail behaviour after any change. Saving verification as code, not as chat history, is what made later cycles confidently non-regressive.

10. Open Items and Recommendations

Everything still open is a credential or a policy decision, not a code gap. The table below is the recommended order of attack: attach the SMTP provider first so invite delivery becomes real end to end, then flip Telegram and TURN with live credentials, then adopt the secret-rotation runbook before the first production tenant is enrolled.

Item	Current state	Recommendation
SMTP provider	EMAIL rows are SIMULATED mirrors in the Notification trail	Attach a real provider via the SMTP_* surface; keep the audit mirror

Item	Current state	Recommendation
Telegram bot token	Env-guarded; SIMULATED audit trail until a token exists	Set TELEGRAM_BOT_TOKEN and CHAT_ID; inline approvals go live (R-05)
TURN credentials	Relay path verified with test env; STUN fallback green	Procure and set TURN_* for restrictive client networks (R-07)
PORTAL_AUTH_SECRET rotation	Static signing secret; per-tenant epoch revocation works	Adopt a rotation runbook; rotation invalidates all sessions by design
Rate-limit store	In-memory maps, single-instance	Swap to the documented Redis adapter for multi-instance deployment
Client onboarding pilot	Invite issuance and first-login verified in tests	Run the full invite-to-first-login journey with one real client

Table 11 - Open items, their state at closeout, and recommended next steps.

The programme closes in the state a product should want: a verified GO verdict that still holds, a ledger that explains every number on every screen, a security model that has already survived its first adversarial review, and a short, explicit list of what stands between the current build and production traffic. The next natural cycle is the SMTP flip and a single real client pilot; nothing else on the list needs to move first.